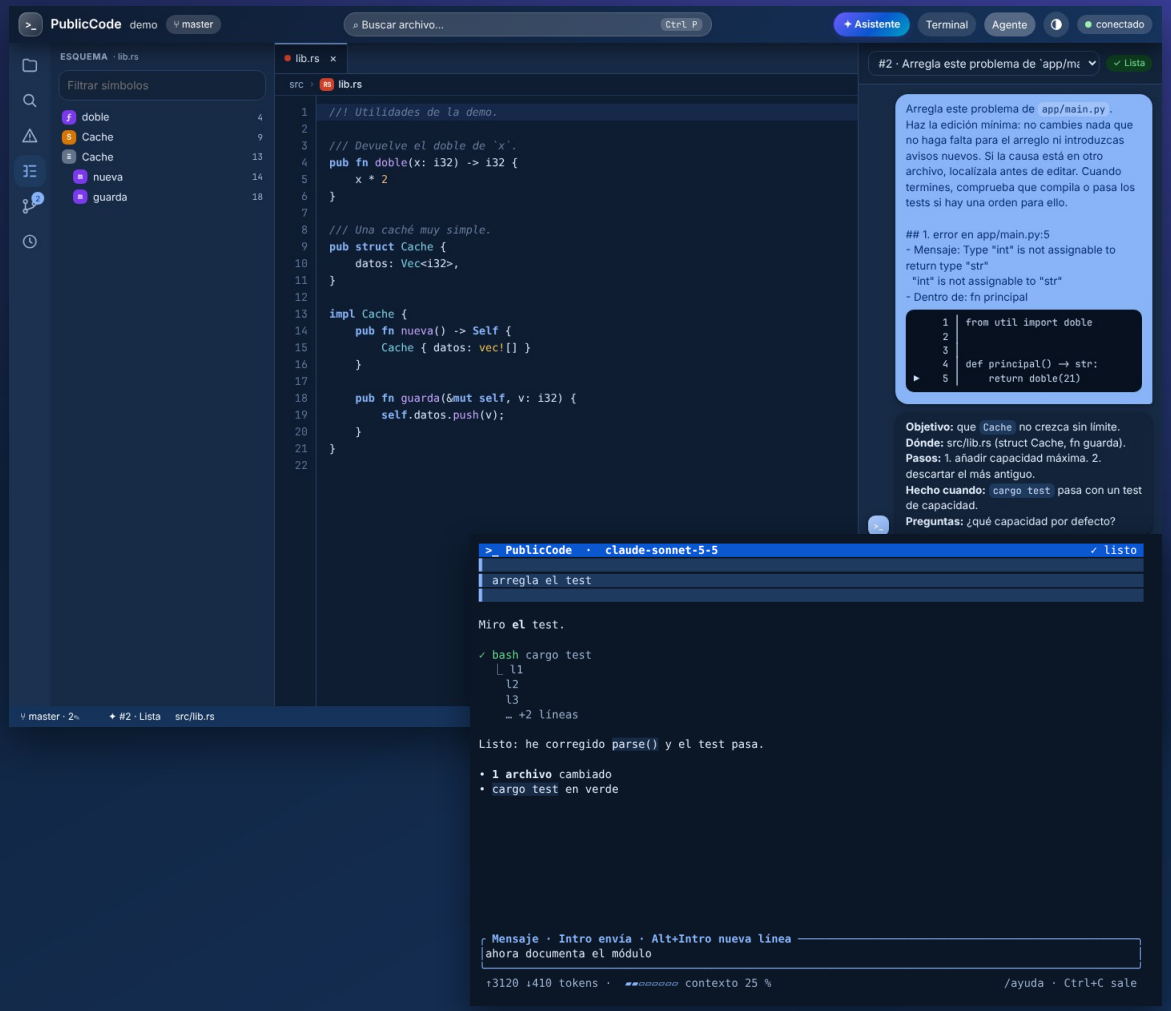
PublicCode

El entorno de programación con agentes que no rompe nada

Editor en el navegador · Terminal · Tu agente y los de siempre (Claude Code, Copilot, Gemini, opencode) · Linux y Windows

v1.1 | Rust · un binario | Apache-2.0



Un entorno completo, no otro chat

PublicCode junta en un solo binario el editor, la terminal y el agente, y le pone alrededor lo que falta para usarlo en serio: reglas que impone el motor, deshacer, revisión y verificación.



1

binario

para Linux y Windows, sin dependencias; Studio corre en tu navegador



4

interfaces

TUI, chat y scripts, Studio y app Android para aprobar desde el móvil



8+

proveedores

Anthropic, Gemini, OpenAI, OpenRouter, Ollama, LM Studio, vLLM, LiteLLM



0

telemetría

solo habla con el modelo que elijas y, al actualizar, con GitHub

Cómo funciona: los clientes muestran, el núcleo decide



Una sesión empezada en la terminal se sigue en Studio o en el móvil: todos hablan con el mismo demonio.

POR QUÉ CAMBIAR

Mejor que VS Code y que opencode

VS Code es un gran editor al que se le añade un agente. opencode es un gran agente de terminal. PublicCode es las dos cosas, con el motor de seguridad en medio.

Frente a VS Code: el agente es parte del editor

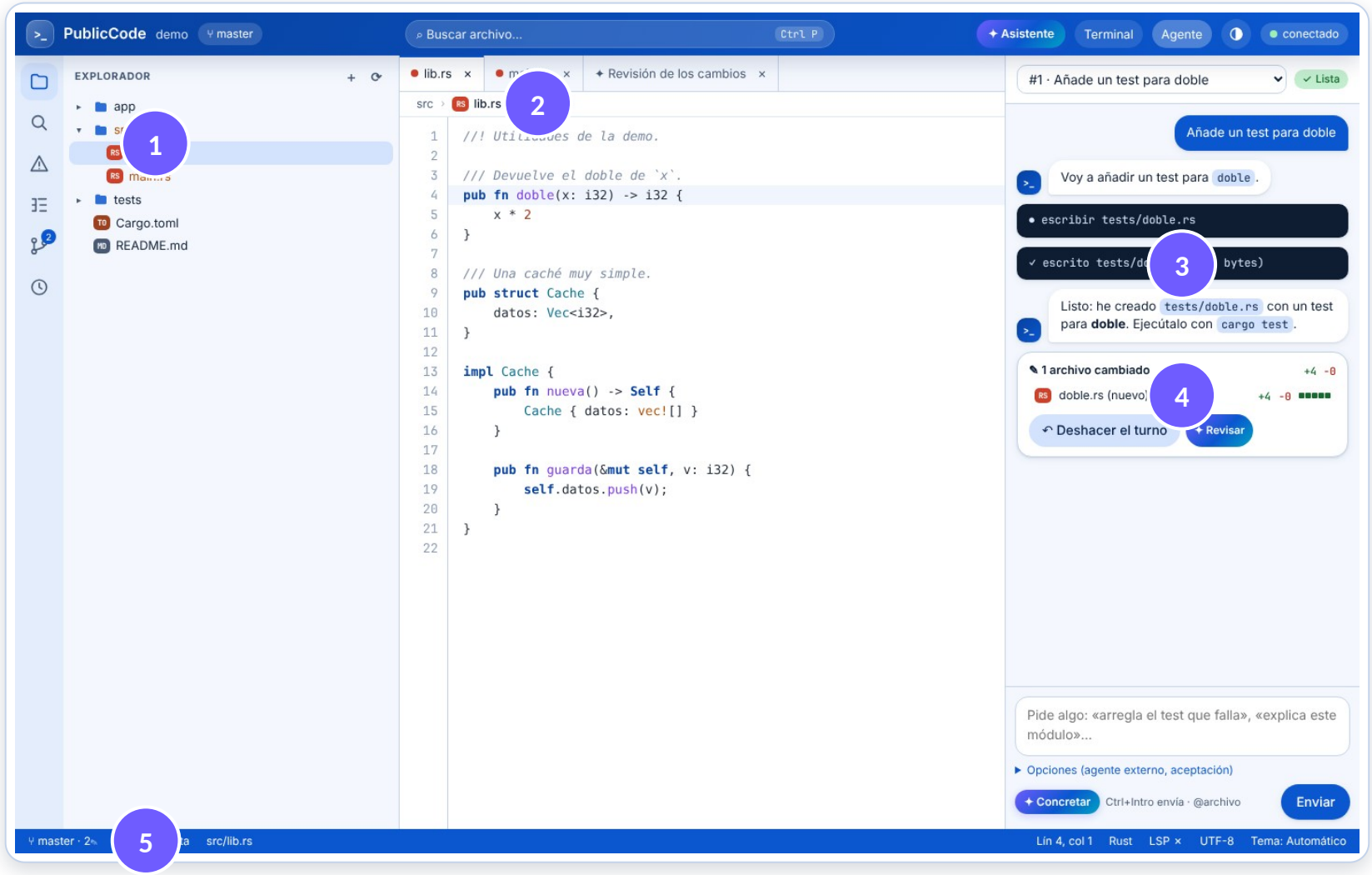
	VS Code	PublicCode
Agente integrado: streaming, aprobaciones y modo plan	con extensión	✓ de serie
Deshacer lo que hizo el agente, turno a turno	según la extensión	✓ ↶ y Actividad
Mensaje de commit generado y revisión antes del commit	con extensión	✓ ✦ Generar · ✦ Revisar
Arreglar un error con su diagnóstico (Problemas, hover, terminal)	—	✓ ✦ Arreglar
Política que impone el motor y auditoría con hash encadenado	—	✓
Modelos en la nube y locales desde una pantalla de Ajustes	con extensiones	✓ 8 plantillas
Aprobar al agente desde el móvil	—	✓ APK
Depurador y extensiones	✓	en el plan (1.x)



Lo que ya usas, también

- Pestañas, resaltado, buscar y reemplazar, Ctrl+P y paleta
- LSP: definición, referencias, hover y errores en línea
- Esquema, migas de pan, Ctrl+T y marcas de git en el margen
- Git con diff y commit, terminal y tareas del proyecto
- Temas claro, oscuro y medianoche; paneles redimensionables

Studio: editor, git, terminal y el agente al lado



1. **Explorador**

   con estado de git e insignias por lenguaje

2. **Migas de pan**

   archivo y símbolo en el que estás

3. **El agente**

   conversación en streaming, con sus tools

4. **Tarjeta del turno**

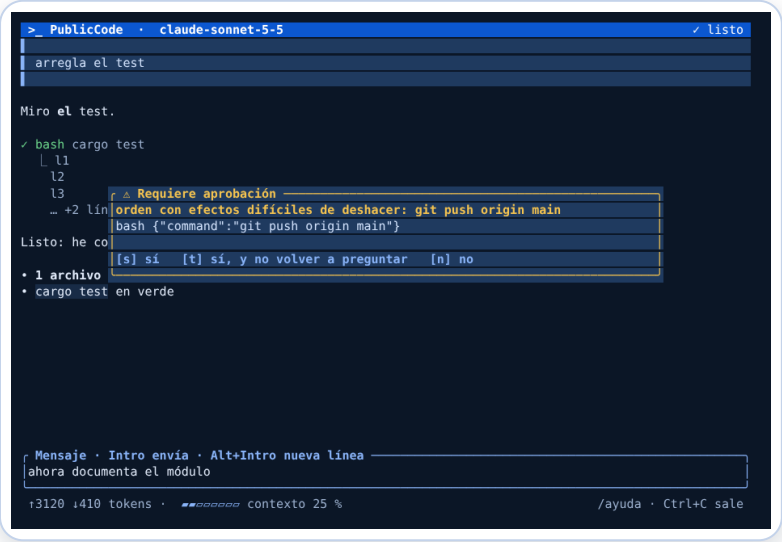
   archivos tocados, +/−, ↶ Deshacer y ✦ Revisar

5. **Barra de estado**

   rama, problemas, agente y modelo

Frente a opencode: lo que pasa alrededor del agente

	opencode	PublicCode
Agente en terminal, modo plan, @archivo, !orden, MCP	✔	✔
Deshacer	✔	✔ también en Studio
Editor completo con LSP, git, esquema y terminal	—	✔ Studio
Claude Code, Copilot, Gemini y opencode como subagentes en sandbox	—	✔ run --backend
Política impuesta por el motor y auditoría encadenada	permisos	✔
Tareas con criterio de aceptación, reintentos e integración	—	✔ pcode flow
Commit generado, revisión y arreglo con diagnóstico	—	✔ ✦
Seguir y aprobar desde el móvil	—	✔ APK



Y en la terminal, igual de cómodo: la TUI pregunta antes de lo destructivo (aquí, un git push).

/deshacer

/recuerda

@archivo

Mayús+Tab plan

FUNCIONES EXTRA

Lo que no tiene ningún otro

Cada función ✦ sale de un paper concreto de ACL o EMNLP 2026 sobre agentes de código.

Asistente ✦: acciones de un clic con el contexto justo

Generar el commit

Resumen por archivo, funciones tocadas, tu estilo y el diff

HierDiff

Revisar los cambios

Errores, alcance, mantenibilidad, tests y seguridad, con veredicto

OmniCode

Arreglar

Desde Problemas, el hover o una orden que falla, con símbolo y código

SHERLOC

Concretar la petición

Objetivo, archivos, pasos, criterio de hecho y preguntas

CodeScout

Explicar

El archivo o la selección, con su esquema

SWE-QA

Escribir tests

En el estilo y el framework del proyecto, y los ejecuta

OmniCode

Un turno del agente, de principio a fin

1



Pregunta si hace falta

❓ ante una petición ambigua, con opciones

Ask or Assume?

2



Localiza

esquema: símbolos y dónde se definen, sin leer todo

RepoNav

3



Edita lo mínimo

punto de control antes de cada escritura

When Models Edit Too Much

4



Comprueba

tests o compilación antes de terminar

Polished but Unresolved

5



Te lo enseña

tarjeta con archivos y +/−

RECAP

6



Deshaces si quieres

↶ el turno entero, ↷ rehacer

How Coding Agents Fail Their Users



Memoria del proyecto en cada turno

/recuerda «los commits, en inglés»: sobrevive a la compactación y el agente solo puede proponer notas que tú apruebas (SLUMP, AMIA).

Seguridad en capas: el prompt no basta

Política del motor

rm -rf /, mkfs, format: se deniegan siempre. git push, reset --hard: se preguntan.

Mini sandbox

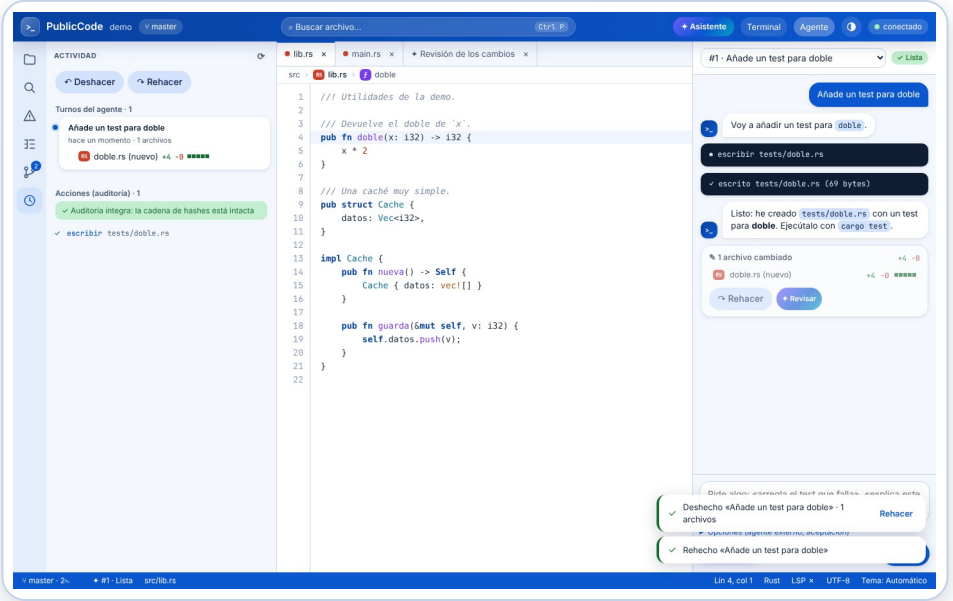
Agentes externos en overlay sin red (Linux) o en git worktree (Windows).

Puntos de control

Cada archivo que escribe el agente se guarda antes: deshacer por turno.

Auditoría encadenada

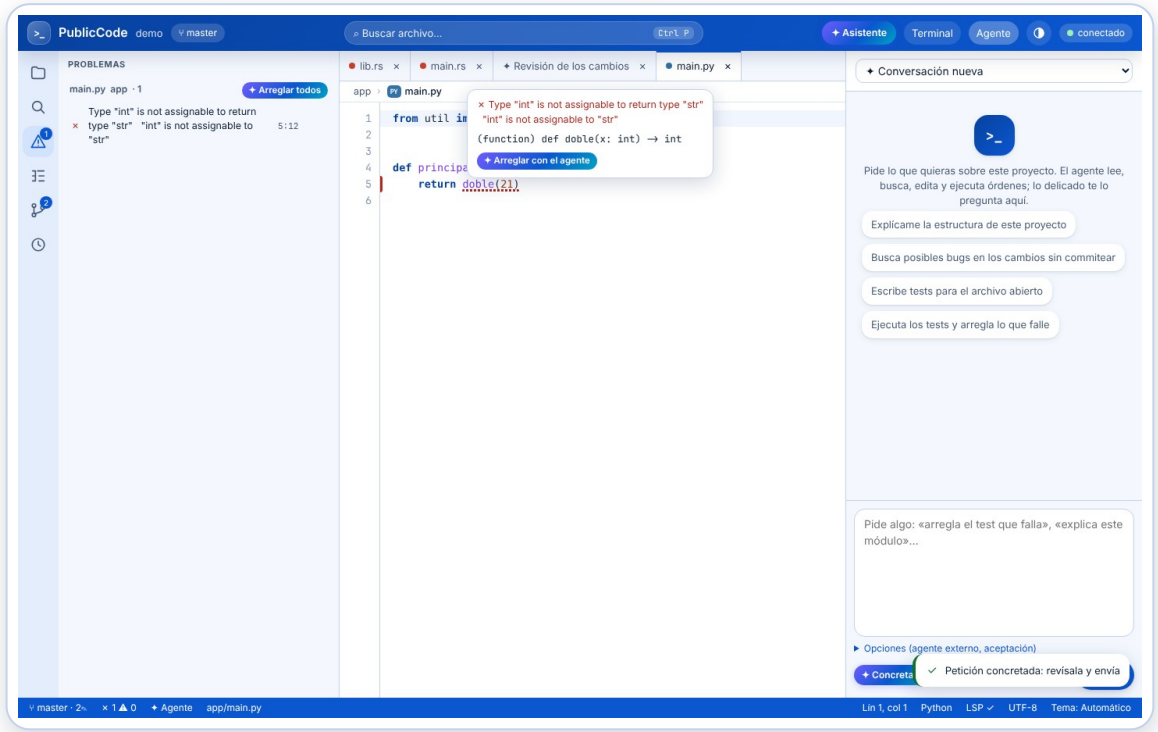
Cada acción, con hash encadenado; Studio comprueba que nadie la ha tocado.



Vista Actividad: cada turno con sus archivos, deshacer y rehacer, y la auditoría con su integridad comprobada.

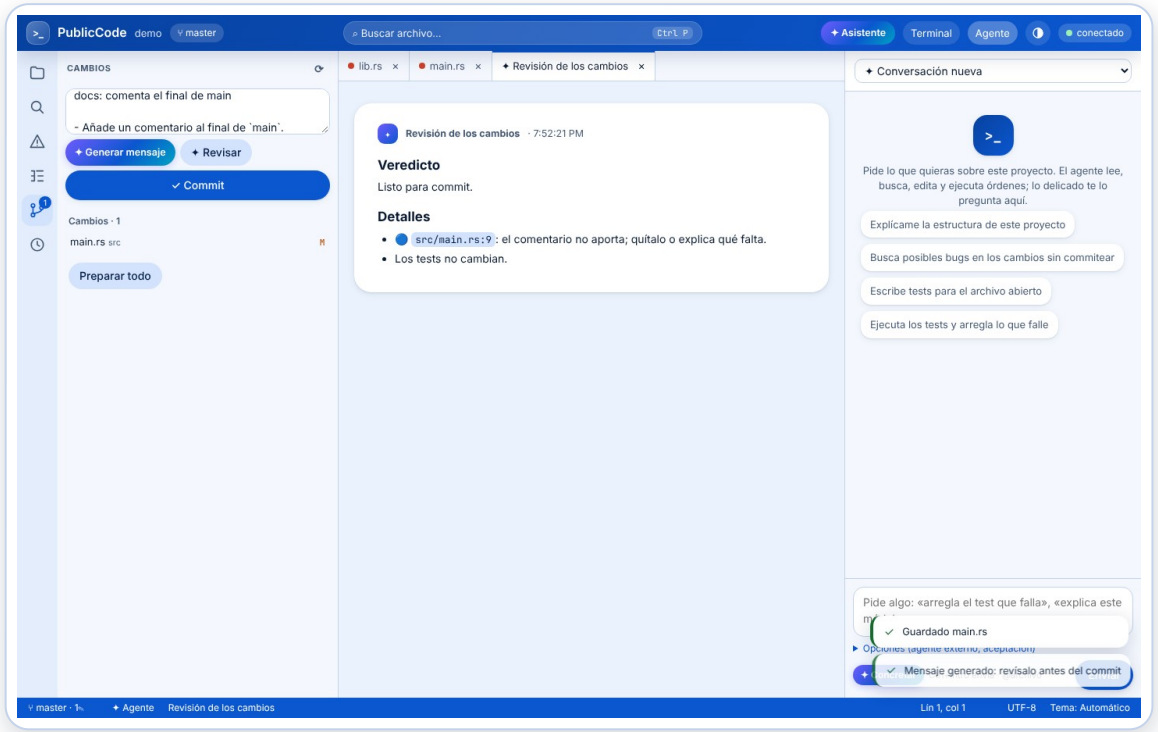
Eval de seguridad en cada CI: 20 casos de inyección de prompts y órdenes destructivas.

Del error al arreglo, y revisión antes del commit



✦ Arreglar con el agente

El error del LSP (aquí con pyright) llega al agente con archivo, línea, función y código alrededor.



✦ Revisar y ✦ Generar mensaje

Antes del commit: revisión con veredicto y el mensaje escrito a partir del diff y de tu estilo.

Lo nuevo de la 1.1: un agente más listo

Localiza antes de leer

tool esquema · RepoNav, SHERLOC

Pregunta si hace falta

como mucho 2 por petición · Ask or Assume?

Recuerda decisiones

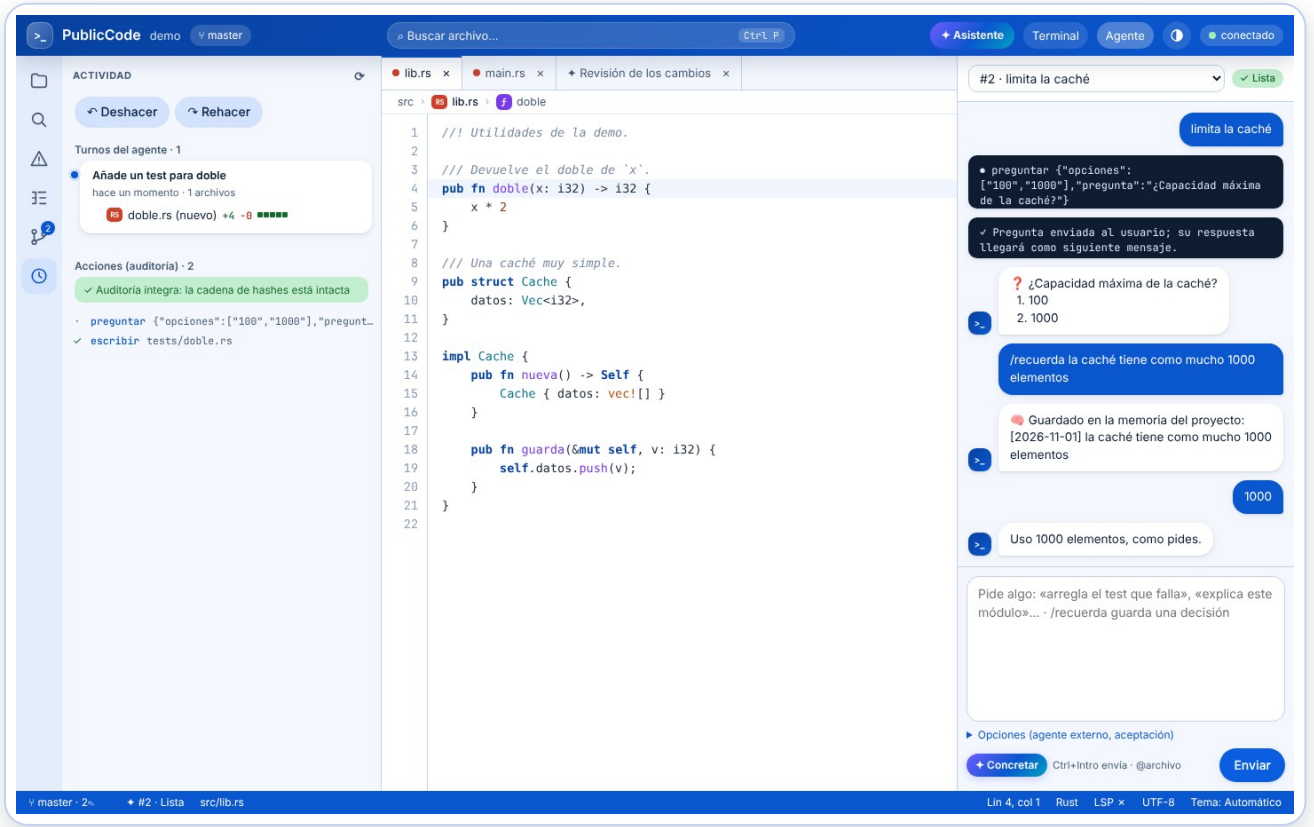
memoria del proyecto · SLUMP, AMIA

Comprueba antes de terminar

Polished but Unresolved

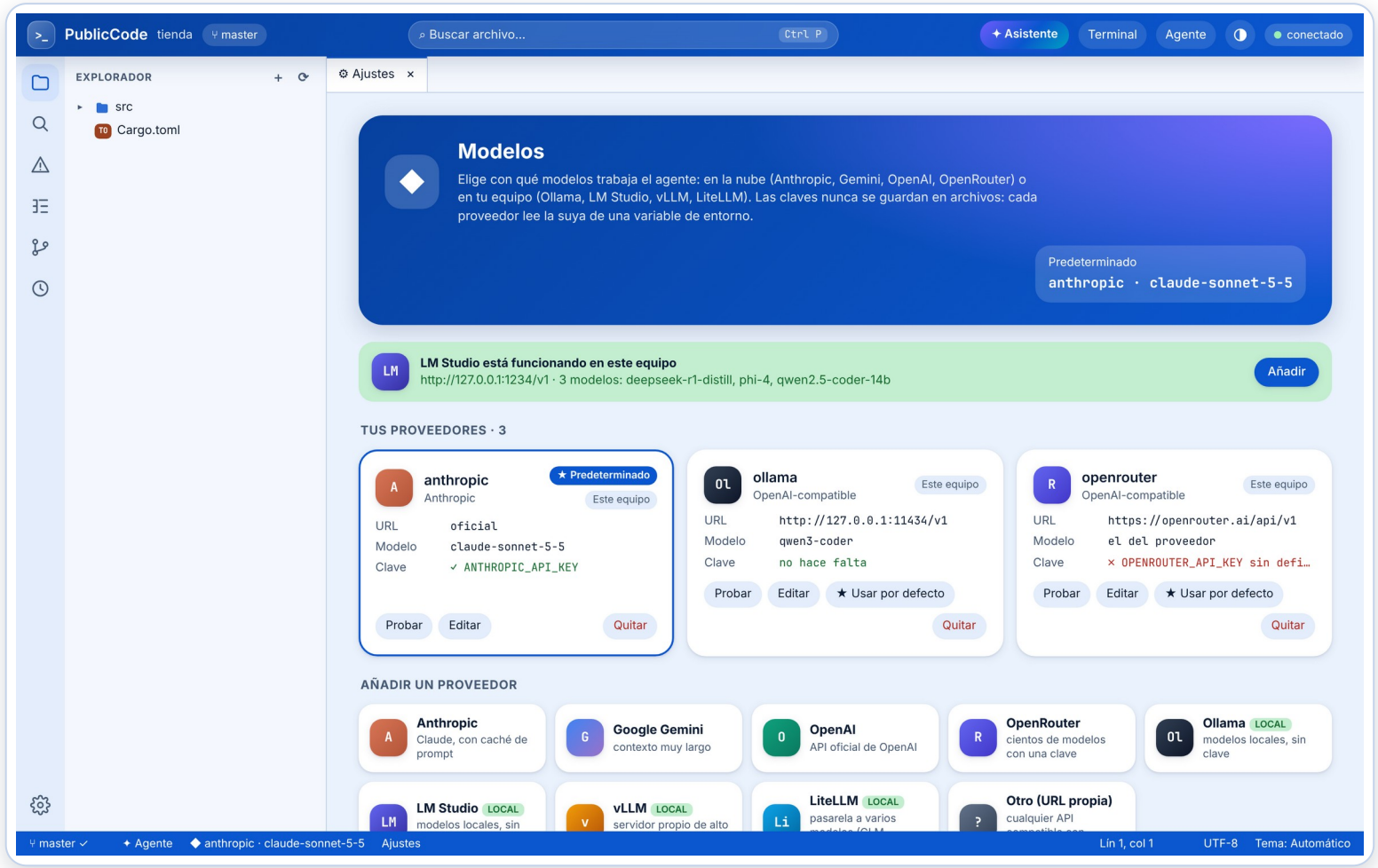
Coste de MCP a la vista

tokens por esquema · ProMCP



El agente pregunta antes de suponer; la decisión se guarda con /recuerda y la lee en cada turno.

Modelos: por configuración o desde Ajustes



8 plantillas y «Otro»

Anthropic, Gemini, OpenAI, OpenRouter, Ollama, LM Studio, vLLM, LiteLLM o cualquier URL compatible

Probar conexión

lista los modelos de esa URL; un clic elige uno

Detecta lo local

Ollama, LM Studio y vLLM en tu equipo

Claves en variables

se guarda el nombre (OPENROUTER_API_KEY), nunca la clave

```
pcode config proveedor ollama
```

```
pcode config proveedores --probar
```

Basado en investigación, no en intuiciones

10.341

papers de ACL y EMNLP 2026 analizados con sus resúmenes

1.189

candidatos relevantes para agentes de código

25+

funciones de PublicCode con su paper detrás

Paper	Qué cambia en PublicCode
SHERLOC (EMNLP)	✦ Arreglar con diagnóstico estructurado
HierDiff (Findings EMNLP)	✦ Generar el mensaje de commit
RepoNav (EMNLP)	esquema, migas de pan y tool esquema del agente
Ask or Assume? (Findings EMNLP)	preguntas de aclaración con presupuesto
SLUMP (Findings EMNLP)	memoria del proyecto que sobrevive a la compactación
When Models Edit Too Much (EMNLP)	ediciones mínimas
ProMCP (Findings ACL)	coste de los esquemas MCP a la vista

Lista completa, con enlaces a la ACL Anthology: docs/papers.md

CÓMO EMPEZAR

Instalado en un minuto

Por usuario, sin administrador, con la suma SHA-256 comprobada. En Linux y en Windows.

Instalar en tres pasos

1

Instala

Una línea; deja pcode en el PATH.

Linux

```
curl -fsSL …/install.sh | sh
```

Windows (PowerShell)

```
irm …/install.ps1 | iex
```

2

Elige el modelo

Una variable de entorno con tu clave, o un modelo local con Ajustes.

```
export ANTHROPIC_API_KEY=…
```

```
pcode config proveedor ollama
```

o Ajustes ⚙️ en Studio (Ctrl+,)

3

Abre tu proyecto

En la carpeta del proyecto: la TUI o Studio.

```
pcode            # la TUI
```

```
pcode studio     # el editor
```

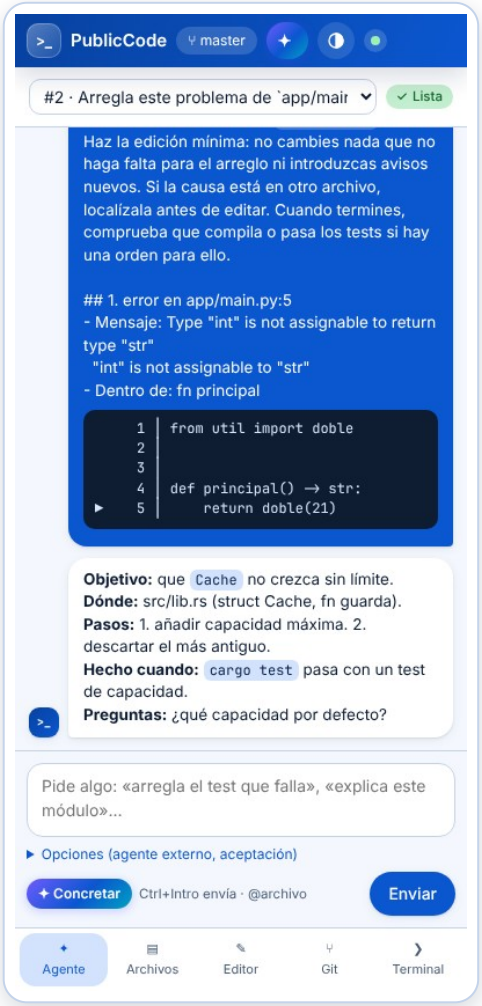
```
pcode doctor     # comprueba
```

Las URL completas están en docs/instalacion.md · Actualizar: pcode actualizar · Android: APK de la release para aprobar desde el móvil

Dónde funciona

Qué	Linux x86_64	Windows 10/11	Android
TUI, chat, Studio, demonio	✔	✔	—
Modelos en la nube y locales, Ajustes	✔	✔	—
Agentes externos en sandbox	✔ **N1: overlay sin red**	✔ **N0: git worktree**	—
Seguir sesiones y aprobar	✔	✔	✔ **APK**
Instalador	install.sh · ~/.local/bin	install.ps1 · %LOCALAPPDATA%	APK de la release

Requisitos: git (recomendado), un navegador para Studio y una clave de modelo o un modelo local. Sin dependencias de ejecución.



Programa con agentes sin perder el control

Editor, terminal y agentes en un solo binario, con política, deshacer, revisión y auditoría.

github.com/jlasherasTracasa/PublicCode

docs/instalacion.md: guía de instalación

docs/usuario.md: guía de usuario

```
$ pcode doctor
$ pcode studio
✓ PublicCode Studio en 127.0.0.1
```